

Agentic RAG Harness Engineering

Teaching language models to look things up, think about what they find — and ship it safely.

Topic 10



WELCOME

What you've signed up for.

- 9 hours total — today's online session (3h) plus one full workshop day (2 × 3h).
- Today: *theory*, with one live demo. No coding.
- Workshop day: *hands-on* build with Claude Code + LangChain.
- Drop questions in chat any time — I'll address them at block boundaries.

| *You don't need to know RAG. We start from zero.*

BY END OF SESSION

What you'll be able to do.

- Explain what RAG is, and why it exists.
- Tell the difference between Classic, Graph, and Agentic RAG — and where enhancement techniques fit on top.
- Recognise when agentic RAG is the right tool — and when it isn't.
- See why harness engineering is what makes any of this safe to ship.
- Be ready to build one on workshop day.

| *This session is the map. The workshop is the territory.*

The next three hours.

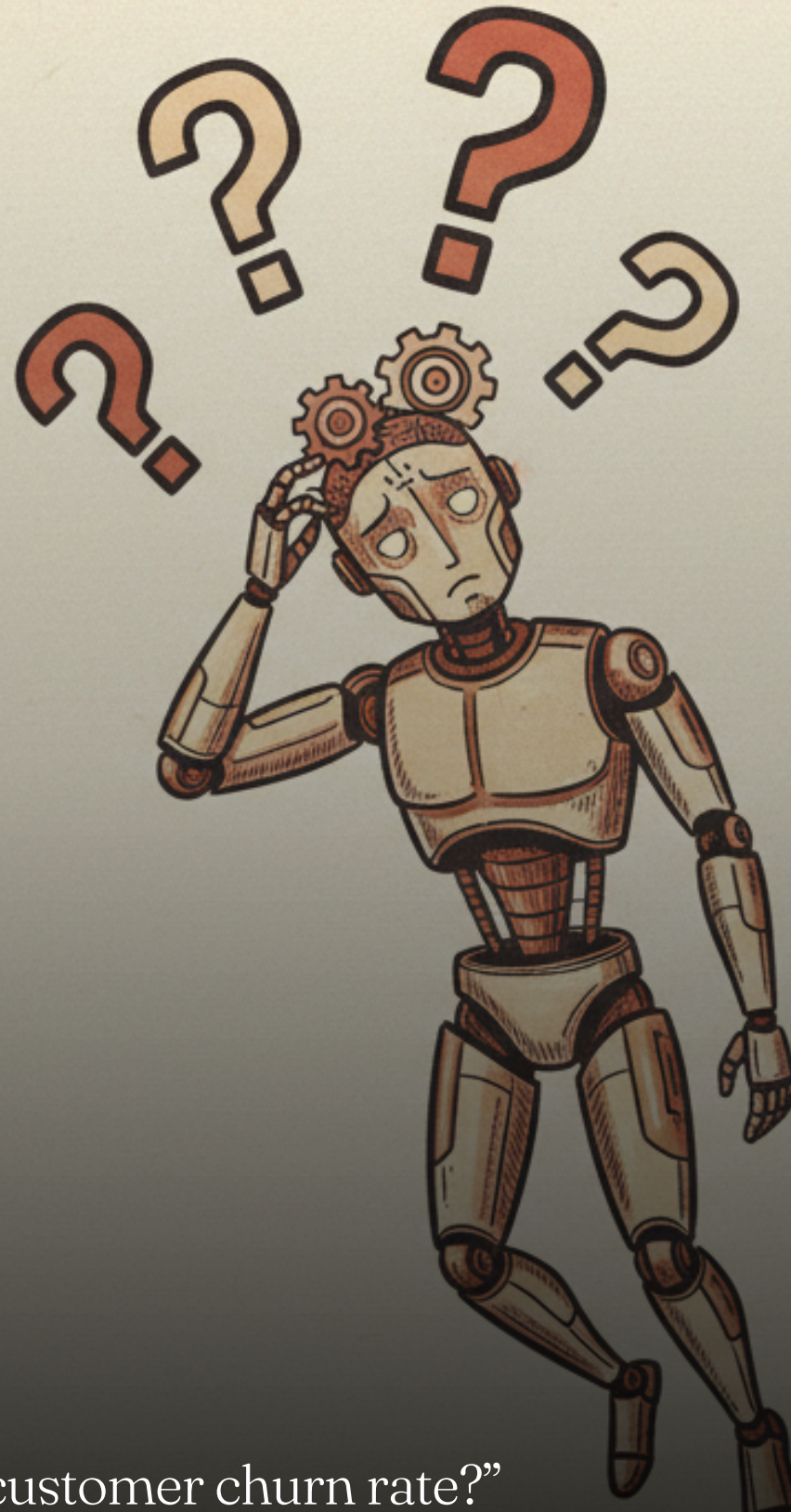
01	Why RAG exists	15 min	02	Classic RAG from first principles	45 min
03	The RAG landscape	25 min	04	Enhancing RAG performance	13 min
05	Where Classic & Graph break	10 min	06	☕ Break	10 min
07	What makes RAG agentic	25 min	08	Core agentic patterns	20 min
09	Micro-break	5 min	10	Live demo (interactive)	20 min
11	Eval, failure modes & harness	22 min	12	Workshop preview	5 min

BLOCK 01

15 MINUTES

Why RAG exists.





BLOCK 1 · A THOUGHT EXPERIMENT

Ask an LLM: “What was our Q1 2026 customer churn rate?”

6 / 101



Four reasons the answer can't be trusted.

01 Hallucination

Fluent but fabricated answers.

02 Knowledge cutoff

Trained on data up to a date. The world moved on.

03 No private data

Your wiki, your PDFs, your database — invisible.

04 No citations

You can't verify where the answer came from.

| *Each of these is fatal in a business setting. RAG addresses all four.*

R is for Retrieval.

- Before the LLM answers, go find relevant information — your documents, your database, your wiki.
- Search by *meaning*, not just keywords — embeddings, coming up in Block 2.
- Output: the top-K most relevant chunks of text.

| *The library trip: find the right pages before anyone writes.*

A is for Augmented.

- Take what was retrieved and add it to the prompt, alongside the user's question.
- No retraining, no fine-tuning — the model just gets better material to work with.
- Prompt = instructions + retrieved chunks + question.

| *The open book lands on the exam desk — the model reads it, it doesn't memorise it.*

G is for Generation.

- The LLM generates the answer, *grounded* in the context it was handed.
- Instructions say: answer from this context — and admit it if the answer isn't in there.
- The answer is checkable: you can point at the exact passage it came from.

| *RAG = give the LLM an open-book exam instead of a closed-book one.*

YOU ALREADY DO THIS

You've done this before — it's called uploading a file.

UPLOAD A PDF, ASK A QUESTION

ChatGPT / Claude / Gemini put the whole file into the prompt · The model reads it and answers — *retrieval, augmentation, generation in one drag-and-drop* · Works great for one document

WHY THAT STOPS WORKING

Context windows have a hard limit — one PDF fits; your whole shared drive doesn't · You can't upload 10,000 pages per question · *RAG's fix: index everything once, retrieve only the few chunks that matter — the prompt stays small, the knowledge doesn't have to.*

The prompt behind the upload.

Answer the question using ONLY the context below.

Context:

{uploaded_document}

Question: {user_question}

Answer:

The LLM doesn't know the answer. It reads and summarizes.

● YOU TELL ME · 60 SECONDS

When would you use RAG — and when would you skip it?

Think of one task in your work where retrieval over documents would genuinely help — and one task where it wouldn't add anything.

| *Drop one of each in chat. Then we'll compare notes.*

When you need RAG — and when you don't.

USE IT FOR

Q&A over private documents · Customer support assistants · Internal research & knowledge search · Anything where citing the source matters

SKIP IT FOR

Pure creative writing · Open-ended brainstorming · Most code generation · Math & pure logical reasoning

Build it, or just use NotebookLM?

USE THE OFF-THE-SHELF TOOL WHEN...

Your data can leave your perimeter · Off-the-shelf retrieval is good enough
· A standalone tool fits the workflow · Time-to-value matters more than control

BUILD YOUR OWN WHEN...

Data residency or compliance forbids upload · You need custom retrieval — hybrid, re-rank, agentic loop · It has to live inside your existing app (Slack, CRM, portal) · Per-seat pricing breaks at 10k+ users · You need a full audit trail of every retrieval

● CHAT CHECK-IN · 60 SECONDS

What's one task in your work where RAG could help today?

One sentence. Be specific — a real document, a real question your team gets asked.

| *Drop it in chat. I'll pick a few to react to.*

Meet Ninja Guru.

- Ninja Van — a logistics company across Southeast Asia.
- Knowledge scattered across a Google Shared Drive — hundreds of Docs, Slides, Sheets.
- Staff kept asking the same things — which policy, which process, who to contact.
- So we built an internal RAG assistant over those docs: *Ninja Guru*.

| Exactly the 'build your own' case — internal data, internal users, grounded answers.

BLOCK 02

35 MINUTES

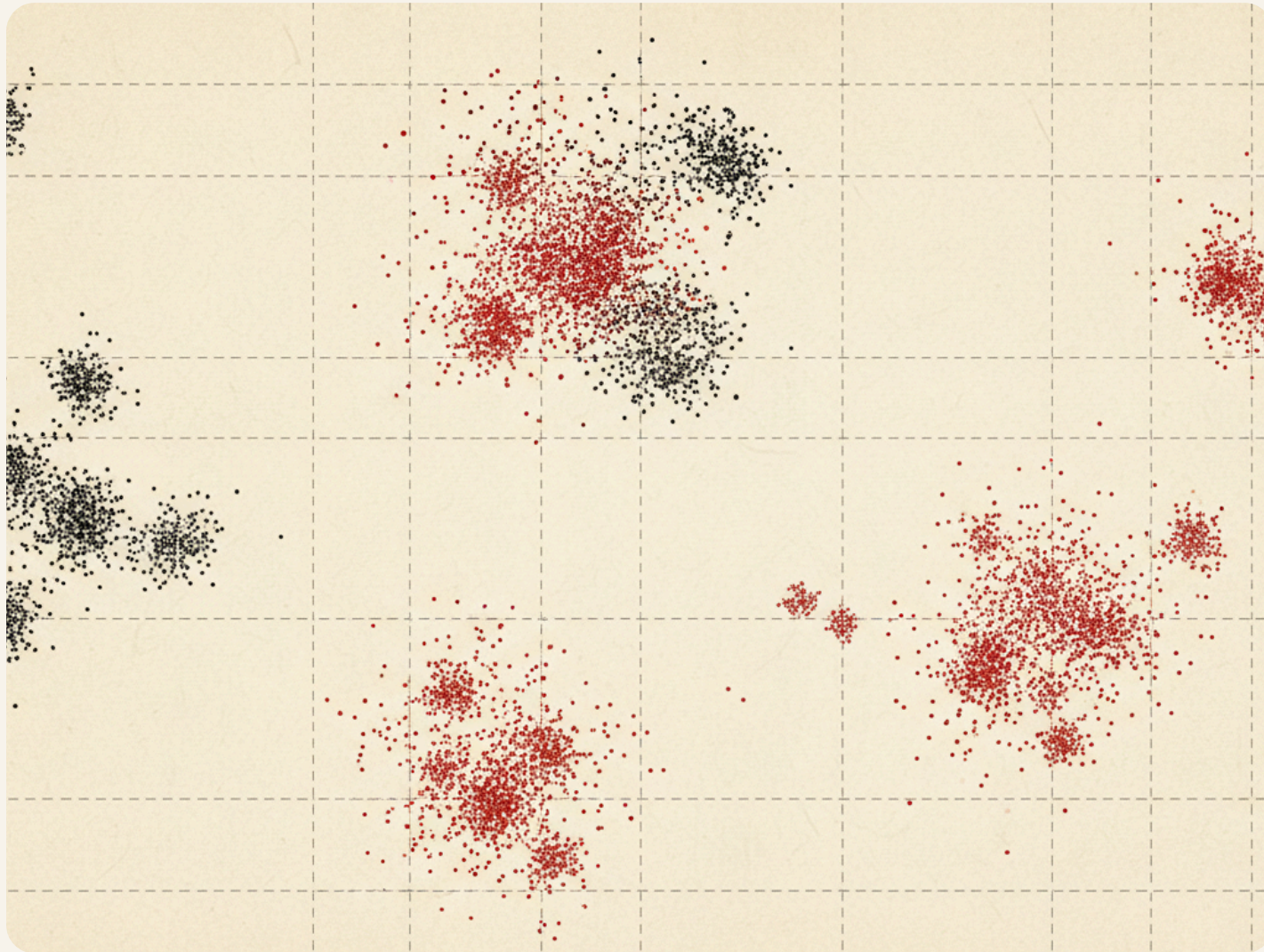
Classic RAG, from first principles.

A MOTIVATING EXAMPLE

How do we find what's relevant?

- A user asks: *"How many days of annual leave do I get?"*
- Your HR handbook: 200 pages.
- Keyword search ("annual leave") may miss "vacation entitlement," "PTO," etc.

| *We need to match by meaning, not just words.*



An embedding is a list of numbers that captures meaning. Texts with similar meanings end up close together in space. Different meanings drift apart.

A VECTOR, IN TWO DIMENSIONS

In 2D, you can actually draw it.

- A vector is just a list of numbers — and a list of numbers is a *point in space*.
- Two numbers, $[3, 4]$: go 3 along the first axis, 4 up the second, and draw an arrow from the origin to that point.
- Add a second vector, $[4, 3]$ — almost the same numbers. Its arrow points in nearly the same direction, so the angle between them is small. *That small angle is what "similar" looks like.*

| *Real embeddings do exactly this — just with 384 or 768 axes instead of 2. Same idea; you simply can't draw it.*

EMBEDDINGS, CONCRETELY

Same meaning → nearby vector.

"I love pizza" → [0.21, -0.45, 0.88, 0.12, ...]

"Pizza is my favorite food" → [0.19, -0.43, 0.85, 0.14, ...] ← close!

"The economy is slowing" → [-0.62, 0.31, 0.04, -0.71, ...] ← far

WHERE DO EMBEDDINGS COME FROM?

A neural network, trained on billions of text pairs.

01 Pre-training

The model sees millions of sentences and learns: words that appear in similar contexts should produce similar vectors.

02 Contrastive learning

Push apart vectors for unrelated text. Pull together vectors for paraphrases, translations, Q&A pairs.

03 The result

Hand the trained model any new text → out comes its 'meaning fingerprint.' A number for every dimension.

No one labelled what each dimension means. The model learned the geometry of meaning.

What's actually inside?

A TYPICAL EMBEDDING

384, 768, or 1536 numbers · Each one between -1 and 1 · No dimension has a human label · Together they form a point in "meaning space"

COMMON MODELS

text-embedding-3-small — 1536 dims · OpenAI · gemini-embedding-001 — 768 dims · Google · all-MiniLM-L6-v2 — 384 dims · open-source · *More dimensions $\neq$ better. Quality depends on what the model was trained on.*

● PREDICT · 30 SECONDS · GUT FEEL

Two opposite sentences. Guess a number.

"I love pizza" vs "I don't love pizza" — similarity (0 = different, 1 = identical) = ?

| *Drop a number between 0 and 1 in chat. You don't need to know the formula — just your gut.*

MEASURING "CLOSE"

Cosine similarity — the angle between two vectors.

```
sim("I love pizza", "Pizza is my favourite food") = 0.92 (near-identical)
```

```
sim("I love pizza", "Pasta is delicious") = 0.71 (related topic)
```

```
sim("I love pizza", "The economy is slowing") = 0.04 (unrelated)
```

Each vector is an arrow from the origin; the angle between two arrows measures similarity. Magnitude doesn't matter — only direction. Range: -1 (opposite) → 0 (unrelated) → 1 (identical).

WHERE EMBEDDINGS STUMBLE

What they often get wrong.

- **Negation.** "Happy" and "not happy" can sit close — the words overlap.
- **Exact terms.** IDs, version numbers, SKUs, error codes — embeddings blur them.
- **Domain jargon.** Acronyms or in-house terms the embedding model never saw.
- **Long passages.** One vector summarising 2,000 words loses nuance — hence chunking.

This is why production systems pair embeddings with keyword search — hybrid retrieval. We'll meet it again in Block 3.5.

Four steps, two phases — index first, query later.

PHASE 1 — INDEXING · STEPS 1-2

Chunk the documents · Embed every chunk and store the vectors · *Done up front, before anyone asks anything — and again whenever documents change.*

PHASE 2 — QUERYING · STEPS 3-4

Embed the question and search · Build the prompt, generate the answer · *Runs live, on every question — against the index you already built.*

Break documents into chunks.

- LLMs and embedding models have input limits.
- Split documents into manageable chunks ($\approx$ 500 words each).
- Chunk too big $\rightarrow$ noisy retrieval.
- Chunk too small $\rightarrow$ lose context.

| *Chunking strategy is one of the biggest knobs in RAG quality.*

STEP 2 OF 4 — INDEXING

Embed every chunk.

- Run each chunk through an embedding model.
- Get one vector per chunk.
- Store every vector in a vector database — Chroma, Pinecone, pgvector...

| *This is the slow, one-time part. Done when documents change.*

STEP 3 OF 4 — QUERYING

At query time, four small actions.

- 01 User asks a question.
- 02 Embed the question using the same model.
- 03 Find chunks whose vectors are closest to the question's.
- 04 Return the top-K nearest chunks (e.g. $K = 5$).

Step 4 — Build the prompt.

Answer the question using ONLY the context below.

Context:

{retrieved_chunks}

Question: {user_question}

Answer:

The LLM doesn't know the answer. It reads and summarizes.

WAIT — ISN'T THIS JUST CHATGPT?

You've done this before — it's called uploading a file.

UPLOAD A PDF, ASK A QUESTION

ChatGPT / Claude / Gemini put the whole file into the prompt · The model reads it and answers — *exactly the Step 4 prompt* · Works great for one document

WHY THAT STOPS WORKING

Context windows have a hard limit — one PDF fits; your whole shared drive doesn't · You can't upload 10,000 pages per question · *RAG's fix: index everything once, retrieve only the few chunks that matter — the prompt stays small, the knowledge doesn't have to.*

PUTTING IT TOGETHER

The classic RAG pipeline.



This is the minimum viable RAG. Everything else builds on top of it.

TWO PHASES

Indexing vs. querying.

INDEXING — ONE-TIME, SLOW

Chunk documents · Embed each chunk · Store vectors · *Runs when your data changes.*

QUERYING — REAL-TIME, FAST

Embed the question · Search for similar vectors · Generate the answer · *Runs per user message.*

WALKING THROUGH IT

"How many leave days do I get?"

INPUT Embed the question.

RETRIEVE Top-3 chunks returned – all from page 14 of the HR handbook.

LLM Send chunks + question to the LLM.

ANSWER You are entitled to 18 days of annual leave per year.

| *The model didn't know the number. It looked it up.*

● CHECK-IN · 60 SECONDS · ONE SENTENCE

In one sentence — what does Classic RAG actually do?

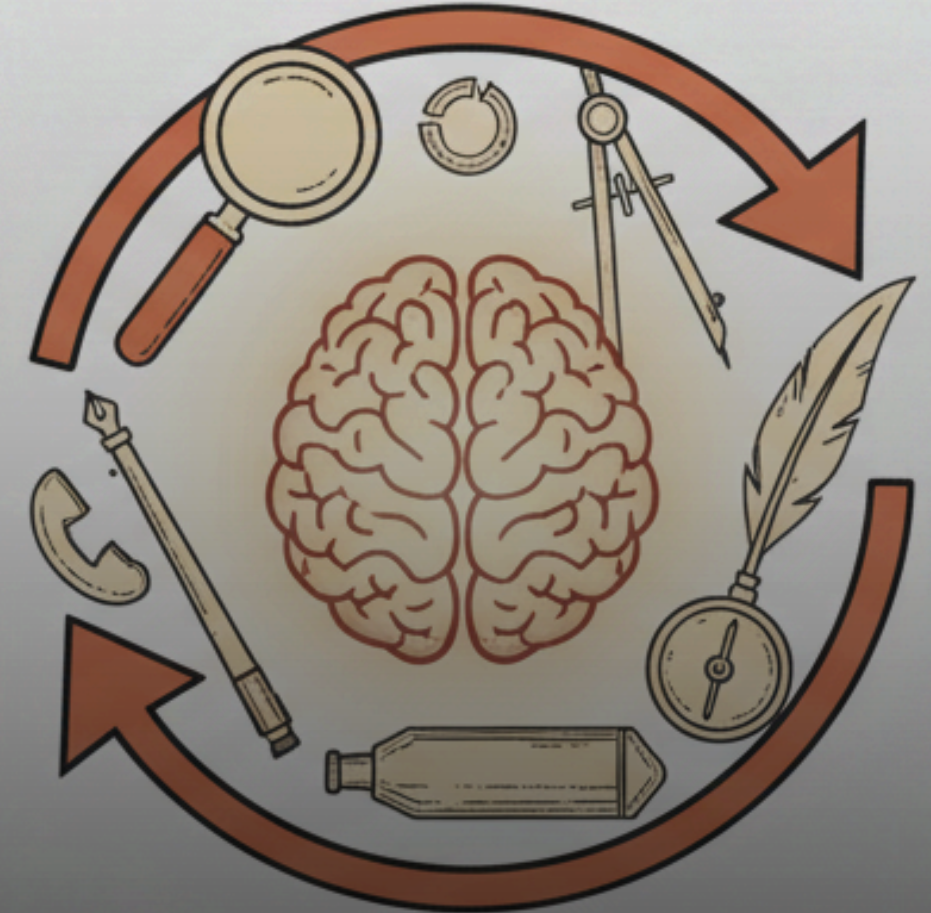
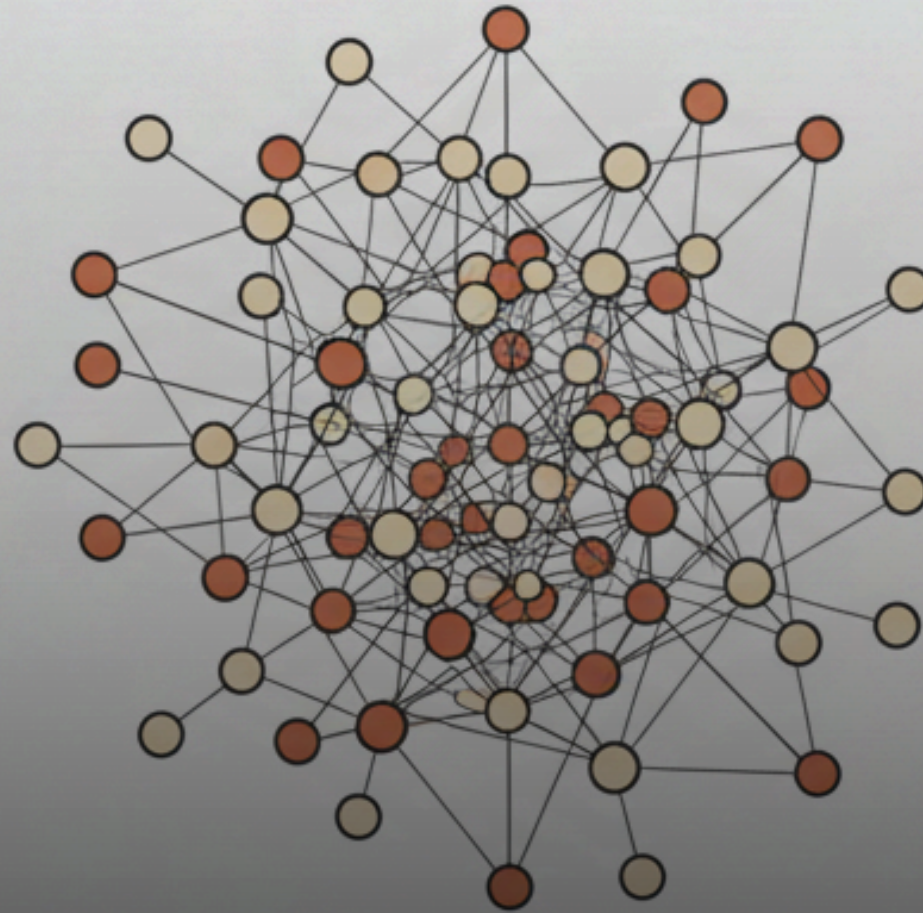
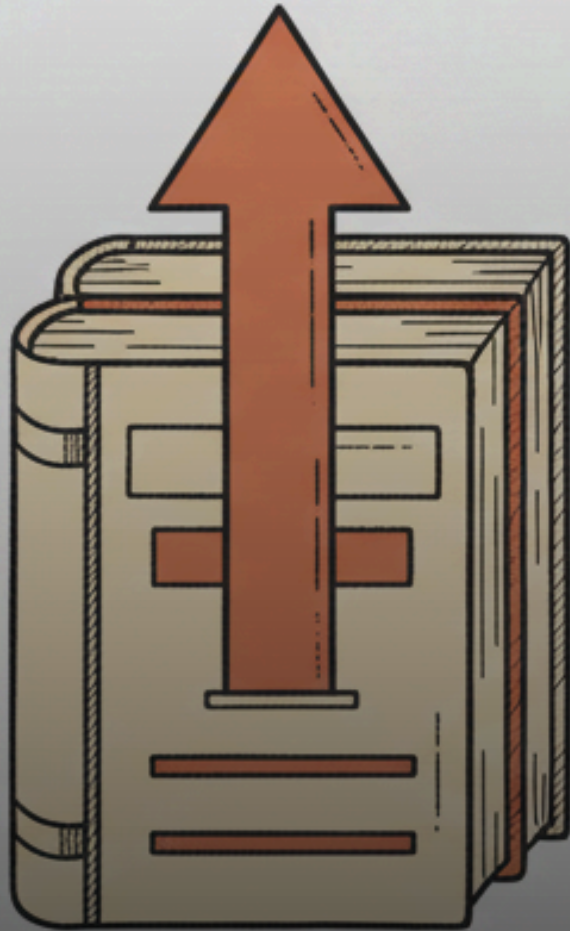
We've spent 45 minutes on this. Before we move on to the landscape — put it in your own words. There's no perfect wording.

| *Drop your version in chat. I'll read a few that capture it well.*

BLOCK 03

25 MINUTES

The RAG landscape: Classic, Graph, Agentic.



BLOCK 3 · THE THREE FLAVORS

Retrieves · Connects · Reasons



NANYANG TECHNOLOGICAL UNIVERSITY | SINGAPORE

FLAVOR 1 — CLASSIC RAG

It retrieves.

- Query → embed → vector DB → top-K → LLM → answer.
- Fast, simple, single-hop. Low cost.
- Falls down on multi-hop: *"Which EU customers also use our API in Asia?"*

| *Great for one-passage lookups. No notion of relationships.*

BLOCK 03

THE LANDSCAPE · FLAVOR 2 OF 3

Graph RAG.



Entities, connected.

- **Nodes** — the things in your world: people, products, places, events.
- **Edges** — typed relationships between them: "employs", "supplies", "is located in".
- Every fact becomes a *triple* — subject, predicate, object: (Acme Corp, supplied_by, BoltCo).

| *A knowledge graph encodes who relates to whom — not just text similarity.*

THE UPFRONT COST

How the graph gets built.

01 Entity extraction

An LLM (or a classic NER model) reads each document and pulls out the entities — people, organisations, places, products, dates.

02 Relation extraction

For each pair of entities in the same context, infer a labelled relationship. Output a triple: (Acme, supplied_by, BoltCo).

03 Disambiguation & merging

Is 'Apple Inc.' the same node as 'Apple'? Resolve duplicates, link to canonical IDs.

04 Store in a graph DB

Neo4j, Memgraph, or even Postgres+pg_age. Now you can query the graph by traversal, not text search.

This pipeline runs once per document. Expensive — that's the trade-off Graph RAG asks you to pay upfront.

GRAPH RAG, AT QUERY TIME

"Which suppliers of our top 3 customers are based in Singapore?"

INPUT Anchor: find top-3 customer nodes by revenue.

QUERY Cypher: MATCH (c:Customer) RETURN c ORDER BY c.revenue DESC LIMIT 3

RETRIEVE [Acme, Globex, Initech]

QUERY For each: MATCH (c)-[:SUPPLIED_BY]→(s)-[:LOCATED_IN]→(:Country {name:'Singapore'})

RETRIEVE [BoltCo, Tessera Pte, Harbour Foods]

LLM Joined, structured results + the question go to the LLM.

ANSWER The three Singapore-based suppliers of our top customers: BoltCo, Tessera Pte, Harbour Foods.

| *The LLM receives joined, structured data — not chunks of unrelated text.*

It connects.

WHAT IT ADDS

Reasoning across entities · Strong for legal, biomedical, customer-360 · Inspired **Microsoft's GraphRAG** (2024) · Heavy upfront cost to build & maintain

WHERE IT STOPS

Still single-shot retrieval · Can't pick a different tool mid-flight · Can't retry if its first answer is weak · Brittle when the schema misses an edge

Structured, not just similar.

CLASSIC RAG — UNSTRUCTURED

A pile of text chunks · Similarity is the only organising principle · Two chunks about the same customer? The system has no idea · *Retrieval* = "find text that sounds like the question."

GRAPH RAG — STRUCTURED

Entities + typed relationships — (Acme, supplied_by, BoltCo) · The structure itself is queryable — follow edges, join facts · Relationships are explicit, not buried in prose · *Retrieval* = "walk the graph and return connected facts."

BLOCK 03.5

13 MINUTES · ORTHOGONAL UPGRADES

Enhancing RAG performance.



THREE UPGRADES THAT STACK ON ANY FLAVOUR

Better retrieval. Same data.

01 Hybrid search

Combine vector similarity with classic keyword search (BM25). Vectors find meaning; keywords find exact terms like model numbers, IDs, error codes.

02 Re-ranking

Retrieve a wider top-K, then a cross-encoder model re-orders by relevance before sending to the LLM.

03 Query rewriting

Use a small LLM to clean up the user's question — fix typos, expand acronyms, add context — before retrieval.

| *These aren't a step between Classic and Agentic. They layer on top of any flavour, including Graph and Agentic RAG.*

Two ways to score relevance.

BI-ENCODER (EMBEDDINGS)

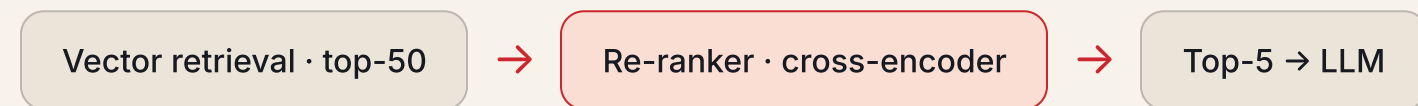
Query and document embedded *separately* — compare the two vectors:
 $\text{score} = \text{cosine}(\text{encode}(q), \text{encode}(d))$ · Fast — ~1ms per comparison, millions pre-indexed · Coarse — meaning squashed into one vector · Good for fetching a broad candidate set

CROSS-ENCODER (RE-RANKER)

Query and document fed to the model *together* — it attends across both, word by word: $\text{score} = \text{encode}(\text{query}, \text{doc})$ · Slow — ~10–50ms per query × doc pair · Can't pre-index — must run at query time · Reads both texts together → much more accurate

THE TWO-STAGE PATTERN

Retrieve wide, re-rank narrow.



- Common re-rankers: **Cohere Rerank** (hosted API) · **BGE Reranker** (open-source) · **Voyage Rerank** (hosted API).
- Often the single biggest quality lift in RAG.
- Adds ~100–300ms latency for a top-50 → top-5 pass.
- No re-indexing required — drop it in front of any retriever.

Bi-encoder gives you recall. Cross-encoder gives you precision. Two stages, one result.

THE MENTAL MODEL

Three axes. Pick any point on each.

AXIS	ONE END	THE OTHER END
Data structure	Text chunks (Classic RAG)	Knowledge graph (Graph RAG)
Retrieval quality	Basic vector search (naïve top-K)	Enhanced (hybrid + rerank + rewrite)
Control flow	Pipeline (fixed steps)	Adaptive loop (Agentic RAG)

Most production systems sit somewhere in the middle of all three. These enhancement techniques aren't a step toward Agentic — they stack on whatever you already have.

What an agent unlocks.

01 Decide whether to retrieve at all

Some questions don't need a lookup — the agent skips it. Saves a round-trip.

02 Decompose multi-hop questions

Break a hard question into sub-queries and answer them in sequence.

03 Route between sources

Vector DB, knowledge graph, web search, internal API — the agent picks the right one for the question.

04 Self-correct on weak answers

If the first retrieval came back empty or off-topic, try a different query — or fall back to admitting it doesn't know.

05 Refuse, gracefully

When your indexed documents genuinely don't cover the question, say so instead of inventing.

| *Not new tools. New behaviour — the LLM gets to decide what to do.*

From pipeline to plan.

CLASSIC & GRAPH — A PIPELINE

Fixed steps, same recipe every query. query → retrieve → generate → answer

AGENTIC — A PLAN

The agent writes its own steps, and changes them as it goes. query → plan → act → observe → re-plan → verify → answer

It reasons.

- A reasoning agent sits in front of all retrieval.
- Vector DB, knowledge graph, web search — all tools the agent picks from.
- A self-evaluation loop checks its own work before answering.

| *Adaptive · multi-step · self-correcting. We unpack the loop in Block 5.*

THE THREE AT A GLANCE

Same goal. Different ambitions.

Classic

'Retrieves'

IMPROVES

Speed of lookup

STRENGTH

Simple, fast

COST

Low

Graph

'Connects'

IMPROVES

Data structure

STRENGTH

Connected context

COST

Medium

Agentic

'Reasons'

IMPROVES

Control flow

STRENGTH

Adaptive reasoning

COST

Higher (latency + tokens)

BLOCK 04

15 MINUTES

Where Classic & Graph RAG break.

THE USUAL SUSPECTS

Four common failure modes.

01 Ambiguous queries

'Tell me about the leave policy.' — which one? for whom?

02 Multi-hop questions

Answer requires combining multiple lookups.

03 Silent hallucinations

Model invents an answer when retrieval misses.

04 Stale or wrong context

Retrieved chunks are off-topic but plausible.

FAILURE MODE #1 — AMBIGUITY

"What's our return policy?"

INPUT The question goes in as-is – embedded exactly as asked.

RETRIEVE Snippet on EU returns. Snippet on B2B returns. Snippet on defective items.

LLM The LLM picks one – or mashes all three together.

ANSWER A confident, technically wrong answer for the user's actual situation.

| *The system couldn't ask: 'EU or US? Consumer or business?'*

FAILURE MODE #2 — MULTI-HOP

"Which products launched after 2024 are still profitable?"

INPUT The whole two-part question goes in as one embedding.

RETRIEVE One retrieval – likely returns half the picture.

ANSWER An incomplete answer the LLM presents as complete.

| *A human would do two lookups. So should the system.*

FAILURE MODE #3 — SILENT HALLUCINATION

"How many medical leave days do contractors get?"

INPUT The question goes in – but the docs only cover full-time staff.

RETRIEVE Retrieval still returns *something* – closest match.

LLM The LLM answers from whatever it was handed.

ANSWER Plausible-sounding fabrication – no flag, no admission.

| *No mechanism to say: 'I checked. This isn't covered.'*

● CHAT POLL · 90 SECONDS

Which of these failures have you actually run into?

Pick one: ambiguous query · multi-hop · silent hallucination · stale context. Or tell me a different one.

| *Drop a one-word answer. We'll regroup right after the break.*

INTERMISSION

10_{min}

Back soon.

BLOCK 05

30 MINUTES

What makes RAG agentic.

● WELCOME BACK · PREDICT

What does an agent give RAG that hybrid search and re-ranking don't?

One word or one phrase. We'll see whose intuition is right over the next 30 minutes.

| *Drop your guess in chat.*

THE SHIFT

From pipeline to loop.

CLASSIC RAG

Fixed steps, in order, every time. $\rightarrow \rightarrow \rightarrow \rightarrow \rightarrow$ *A pipeline.*

AGENTIC RAG

The agent decides what to do next, based on what it just saw. $\rightarrow \rightarrow \hookrightarrow \rightarrow \rightarrow$
 $\hookrightarrow \rightarrow$ *A loop.*

THE CORE IDEA

An agent = LLM + tools + a loop.

- The LLM doesn't just answer — it plans, acts, observes, and re-plans.
- Tools are how it reaches outside itself.
- The loop runs until the task is done.

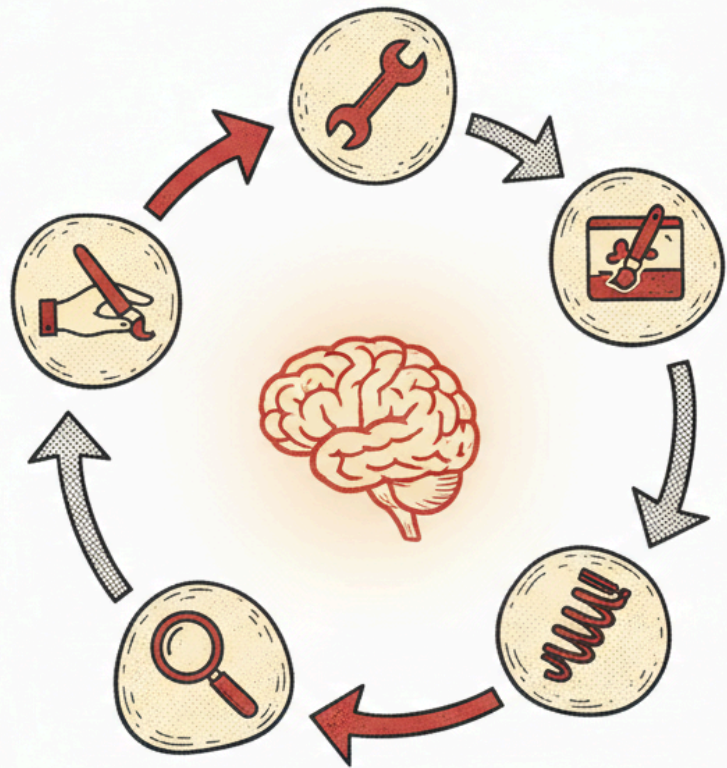
| *A pipeline executes. An agent decides.*

TOOLS

Any function the LLM can call.

- Search the web. Query a vector DB. Hit an internal API. Run a calculator.
- The LLM emits a structured call: `search_docs("leave policy")`
- Your code executes it — the result goes back into the LLM.

| *Tools are how the LLM reaches outside itself.*



Reason → Act → Observe → Repeat. This pattern is called ReAct — the foundation of most modern agents.

THE REFRAME

Retrieval is no longer a step. It's a tool.

CLASSIC

Retrieval is *step 2 of a fixed pipeline*. The system always retrieves, the same way, every time.

AGENTIC

Retrieval is *one tool among several*. The agent decides whether to retrieve, which source, and how many times.

● PREDICT · 30 SECONDS · A / B / C / D

Same hard question. What's the agent's first move?

"Which products launched after 2024 are still profitable?" — A. Search the docs immediately · B. Decompose the question into sub-parts · C. Ask a clarifying question · D. Reject — too vague

| Drop A, B, C, or D in chat. Gut feel — we're about to watch the trace.

THE SAME HARD QUESTION, AGENTIC STYLE

"Which products launched after 2024 are still profitable?"

REASON I need a list of products launched after 2024.

ACT `search_docs('product launches 2024 2025 2026')`

OBSERVE List of 8 products.

REASON Now I need profitability for each.

ACT `query_db('profitability of products X, Y, Z, ...')`

OBSERVE Profitability numbers.

REASON I have enough. Answer now.

ANSWER Products A, C, and F are still profitable.

| *Two retrievals, planned by the agent. Classic RAG would have done one.*

CLOSING THE LOOP

Self-evaluation.

- After drafting an answer, the agent asks itself: *"Is this actually grounded in what I retrieved?"*
- If no → try again with a different query, or admit it doesn't know.
- This is what the "Self-Evaluation" stage in the diagram is doing.

| *The agent doesn't trust its own first draft.*

SIDE BY SIDE

What agents actually add.

Classic RAG

DECIDE WHETHER TO RETRIEVE
No — always retrieves

MULTIPLE RETRIEVALS
No — one shot

ROUTE BETWEEN SOURCES
No — one index

SELF-CHECK ANSWER
No

COST / LATENCY
Low

Agentic RAG

DECIDE WHETHER TO RETRIEVE
Yes — agent decides

MULTIPLE RETRIEVALS
Yes — as many as needed

ROUTE BETWEEN SOURCES
Yes — chooses tool

SELF-CHECK ANSWER
Yes

COST / LATENCY
Higher

Trade-off

When is the cost worth it?

WORTH IT
Hard questions, accuracy > speed, multiple sources

NOT WORTH IT
Simple FAQ, tight latency SLA, fixed pipeline works

BLOCK 06

25 MINUTES

Five patterns to know.



The agentic RAG toolkit.

01 Query routing

Pick the right source.

02 Query rewriting / decomposition

Make the question retrievable.

03 Multi-hop retrieval

Chain lookups.

04 Self-reflection / corrective RAG

Grade what you got.

05 ReAct

The loop that holds it all together.

These aren't competing options. Real systems combine them.

PATTERN 1 — QUERY ROUTING

Pick the right source.

- Multiple indices: HR docs, product docs, sales DB.
- A router LLM decides which to query for this question.
- Cuts noise from searching irrelevant corpora.

| *Don't ask the kitchen about the books in the library.*

Make the question retrievable.

USER ASKS

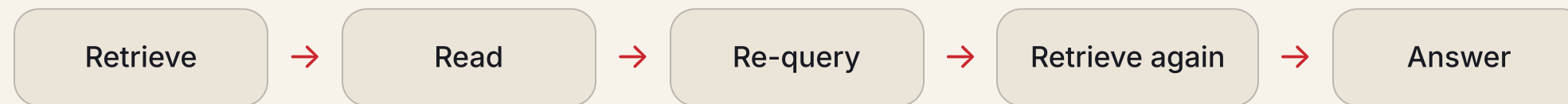
"Compare our 2025 and 2026 leave policies."

AGENT DECOMPOSES

"2025 leave policy" · "2026 leave policy" — *better queries* →
better retrieval → *better answers*.

PATTERN 3 — MULTI-HOP RETRIEVAL

Chain lookups based on what you just learned.



The second query is shaped by what the first one revealed.

Grade your own retrievals.

- After retrieval, the agent rates each chunk: relevant / partial / irrelevant.
- If retrieval was poor, re-query — or fall back to a different tool (web search).
- The agent doesn't trust its tools blindly.

| *A second pair of eyes — except they're the same model, in a different prompt.*

In practice, patterns stack.

- Route the query (1)
- Decompose it (2)
- Do multi-hop retrieval (3)
- Self-grade the results (4)
- All inside a ReAct loop (5).

| *Patterns are LEGO bricks, not menu items.*

INTERMISSION

5^{min}

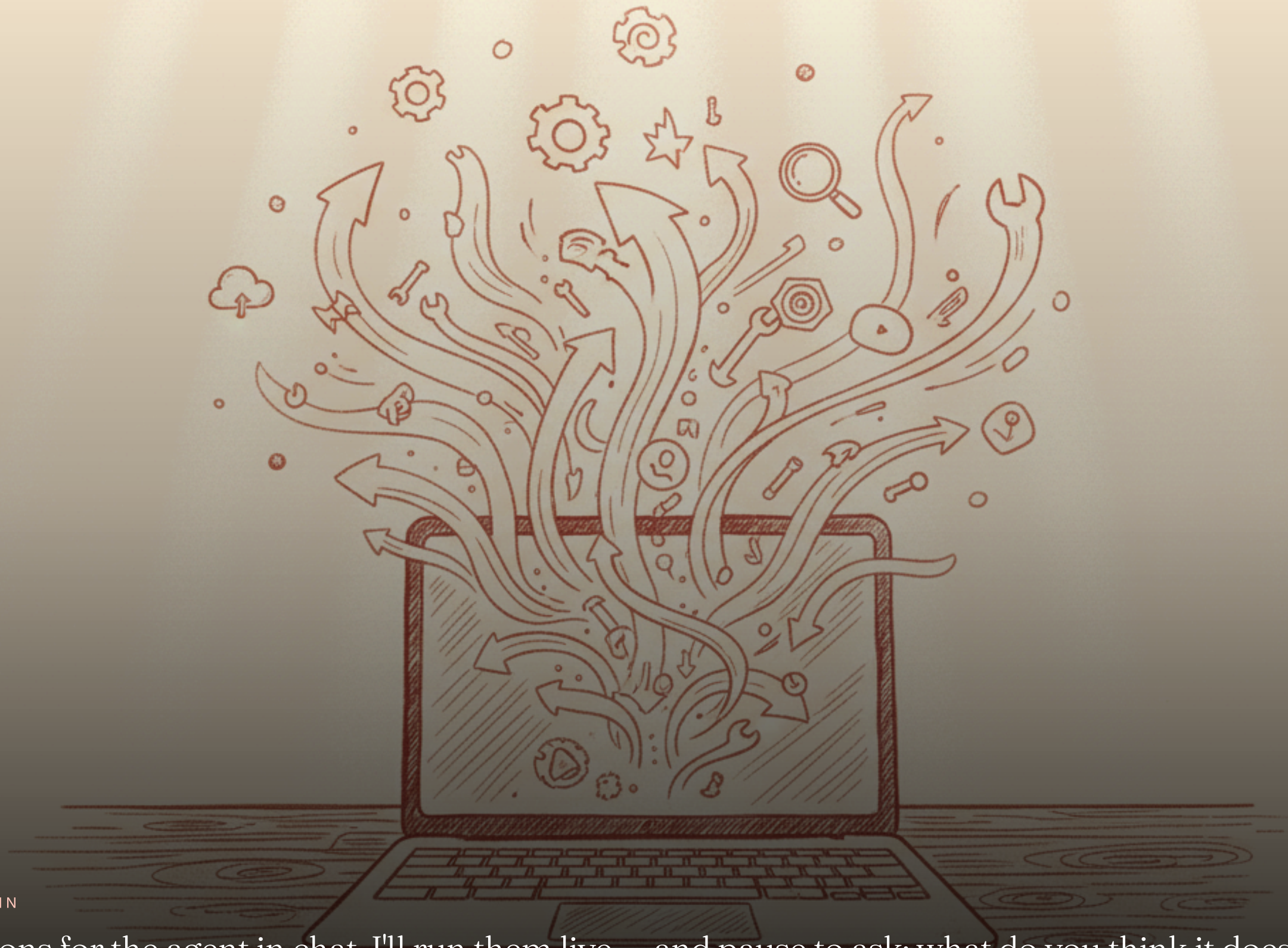
Back soon.

BLOCK 07

25 MINUTES · INTERACTIVE

Live demo.





WATCH — AND JOIN IN

Submit questions for the agent in chat. I'll run them live — and pause to ask: what do you think it does next?

83 / 101



WHAT WE'LL SHOW — TOGETHER

The demo path.

01 The document set

A small set of indexed documents — preview before we ask anything.

02 The agent's tools

Retriever plus an optional second source.

03 A simple question

From me. Watch a single retrieval.

04 Your question

Pulled from chat. Predict the trace before I press enter.

05 An unanswerable one

Together — to see self-correction or graceful refusal.

| *Drop questions in chat any time during the demo.*

BLOCK 08

17 MINUTES

Evaluation, failure & the harness.



*“If you can't measure it, you can't
improve it.”*

— A TIMELESS ENGINEERING PRINCIPLE

RAG has two halves. So does evaluating it.

RETRIEVAL

Did the system find the *right chunks*? · *If retrieval misses, the LLM has nothing to work with — and no amount of prompt-tuning will save you.*

GENERATION

Did the LLM *use them well*? · *Even with good chunks, the model can drift, miss the question, or invent facts.*

Faithfulness & answer relevance.

FAITHFULNESS

Is the answer *supported* by the retrieved context? · *Fails when the model invents facts beyond what it read.*

ANSWER RELEVANCE

Does the answer actually *address* the question? · *Fails when the model is technically correct but unhelpful.*

FAITHFULNESS × RELEVANCE

Four combinations. One is dangerous.

1 · FAITHFUL + RELEVANT

The ideal.

Grounded in the context, and answers the question.

2 · FAITHFUL + IRRELEVANT

Correct, but off-topic.

True to the context — just not what was asked.

3 · UNFAITHFUL + RELEVANT

Sounds right. Isn't.

On-topic, plausible — and invented.

4 · UNFAITHFUL + IRRELEVANT

Obvious failure.

Wrong and off-topic — at least it's easy to spot.

Quadrant 3 is the one that ships — it doesn't look like a failure, it looks like an answer.

Context recall & context precision.

CONTEXT RECALL

Did retrieval get *all* the relevant chunks? · *Fails when chunks that would have answered the question were never retrieved. The LLM can't use what it never saw.*

CONTEXT PRECISION

Were the retrieved chunks *actually relevant*? · *Fails when noise dominates signal. Off-topic chunks dilute the context and can mislead the model.*

THE GOLDEN SET

Twenty questions, repeated forever.

- A small fixed set of test questions with expected behavior.
- Each question has a known correct answer (or a known correct refusal).
- Run all four metrics against it after every change.
- Even 20–30 questions catches most regressions.

| *We'll build one in the workshop.*

HOW DO YOU ACTUALLY SCORE THESE?

LLM-as-judge.

- A strong LLM (often GPT-5, Claude 4, Gemini 3) reads *question + context + answer* and outputs a score.
- Faster and cheaper than human reviewers. Runs on every commit, on hundreds of questions, in seconds.
- Surprisingly accurate — judge models agree with human reviewers ~80–90% of the time on most metrics.
- Watch for: judge bias (LLMs prefer their own style), prompt sensitivity, and judging your own model with itself.

| *LLM-as-judge handles the bulk. Keep a small slice for human review — high-stakes domains, edge cases, calibration.*

DON'T BUILD THIS FROM SCRATCH

Evaluation frameworks you'll meet.

01 Ragas

Open-source, batteries-included. Implements the four metrics above plus a dozen more. Most popular starting point.

02 DeepEval

Pytest-style. Write evals like unit tests. Strong CI integration.

03 TruLens

Tracing plus eval. Lets you record real production traces and replay them against new metrics.

04 Phoenix (Arize)

Observability + eval. Strong choice when you also need production monitoring.

05 LangSmith

LangChain's hosted eval service. Deeply integrated if you're already on the LangChain stack — which we will be in the workshop.

| *In the workshop we'll use Ragas to grade our golden set, with LangSmith for the trace UI.*

FOUR FAILURES TO DIAGNOSE

When agentic RAG misbehaves.

01 Retrieval failure

Top-K doesn't contain the answer.

02 Faithfulness failure

Retrieval was good — answer drifts from it.

03 Loop failure

Agent gets stuck retrying or wanders off-task.

04 Cost blowout

Too many loop iterations — tokens explode.

| *Different failures need different fixes. Diagnose before you treat.*

THE OTHER HALF OF THE TITLE

Harness engineering.

- Every failure mode we just listed needs an engineering answer, not just a prompt.
- Guardrails — limit which tools the agent can call, and how often.
- Tracing — see every reason / act / observe step, in order.
- Cost & loop limits — stop the agent before it spirals.
- Eval hooks — run the golden set on every change, automatically.

| *Agentic RAG works in slides. The harness is what makes it work in production.*

Knowledge agents keep the corpus fresh — and correct.

A RAG system is only as good as its documents. Stale or conflicting files produce confident, wrong answers — so background *knowledge agents* tend the corpus continuously.

- Watch the Shared Drive — Docs, Slides, Sheets — for changes.
- Re-chunk and re-index automatically.
- Flag stale, duplicate, or conflicting information.
- Keep every answer traceable to a current source.

A REMINDER

When not to use agentic RAG.

- The question is simple. Enhanced retrieval (hybrid + re-rank) is faster and cheaper.
- Latency budget is tight (chatbots with <1s SLA).
- You can't afford the extra LLM calls.
- Your document set is small and structured — direct DB query may be better.

| *Agentic RAG is powerful. Power is not always what you need.*

What you'll build.

MORNING · 3H

Classic RAG baseline · Wrap retrieval as a tool · First agent — with a harness around it

AFTERNOON · 3H

Routing + multi-hop · Self-correction · Evaluation, tracing & iteration

BEFORE WORKSHOP DAY

Pre-workshop checklist.

- Install Claude Code (link to follow in chat).
- Python 3.11 or newer installed.
- Node.js 18 or newer installed — Claude Code runs on it.
- Claude account ready (for Claude Code sign-in).
- Clone the workshop repo.

| *Instructions in chat after class — and on the course page.*

WHAT WE COVERED TODAY

Today, in six lines.

- RAG = retrieval + LLM, to ground answers in your data.
- Three flavors: Classic (retrieves), Graph (connects), Agentic (reasons).
- Enhancing RAG performance — hybrid search, re-ranking, query rewriting — is often enough.
- An agent = LLM + tools + loop. Five patterns combine inside ReAct.
- Measure with faithfulness + relevance against a golden set.
- Harness engineering is what makes any of this safe to ship.

| *You now have the map. Workshop day, we walk the territory.*



QUESTIONS?

See you on workshop day. Drop questions in chat — or save them for the workshop.

101 / 101

